

Advanced Image Processing

Assignment 1 Report

Tejash More
SR: 27077

February 1, 2026

1 Question 1: Local Binary Pattern (LBP) for Texture Analysis

This section details the implementation and evaluation of texture classification using Local Binary Patterns (LBP). We compare the performance of Standard LBP (SM-LBP) and Rotation Invariant LBP (RI-LBP) on the Describable Textures Dataset (DTD), analyzing the impact of image rotation.

1.1 Methodology

1.1.1 Standard LBP

Standard LBP encodes the relationship between a center pixel g_c and its neighbors g_p on a circle of radius R :

$$LBP_{P,R} = \sum_{p=0}^{P-1} s(g_p - g_c) \cdot 2^p \quad (1)$$

Where the threshold function $s(x)$ is defined as:

$$s(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ 0 & \text{if } x < 0 \end{cases} \quad (2)$$

This results in a feature space of 256 distinct integer values (0 – 255).

1.1.2 Rotation Invariant LBP (RI-LBP)

Proposed in [1], RI-LBP achieves invariance by treating the binary code as a circular string and normalizing it to its minimum integer value:

$$LBP_{P,R}^{ri} = \min\{ROR(LBP_{P,R}, i) \mid i = 0, \dots, P-1\}$$

Where $ROR(x, i)$ denotes a circular bitwise right-shift of x by i positions.

This reduces the feature space significantly. For $P = 8$, the unique patterns reduce from 256 to 36.

1.2 Implementation

- **Feature Vector:** A normalized histogram of LBP codes (256 bins for Standard, 36 bins for RI) was computed for each image.
- **Classifier:** A 1-Nearest Neighbor (1-NN) classifier using Euclidean distance.
- **Dataset:** Experimentation was done on the full dataset, that is, all 47 classes, as well as on a small subset of 5 random classes.

1.2.1 Feature Visualization

We verify the implementation by visualizing the feature maps and Histogram extracted from a sample 'veined' texture, for Standard LBP as shown in fig: 1 and for Rotational Invariant LBP as shown in fig: 2

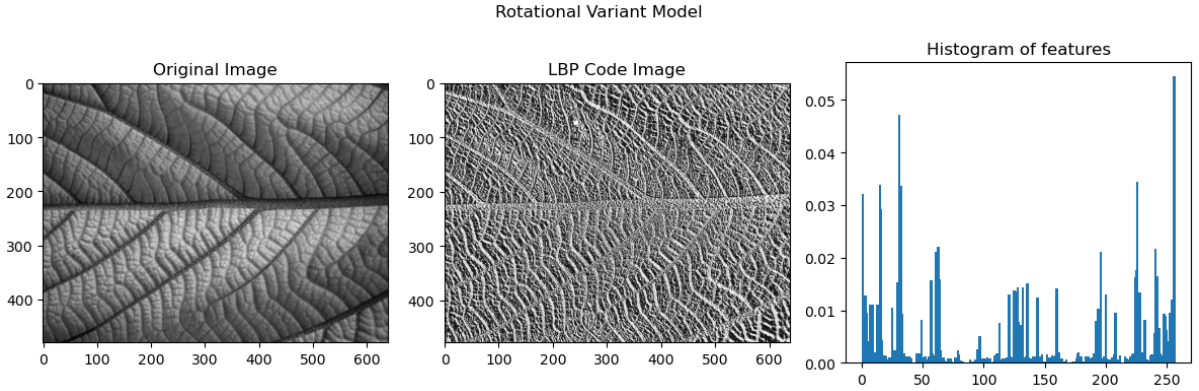


Figure 1: **Standard LBP Feature Extraction.** Left: Original Image. Center: Standard LBP Map (0 – 255). Right: Feature Histogram.

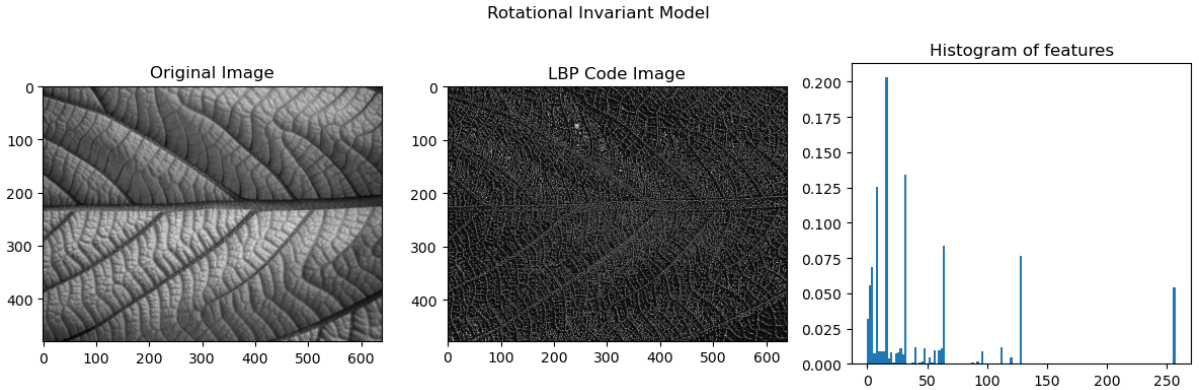


Figure 2: **RI LBP Feature Extraction.** Left: Original Image. Center: Rotational Invariant LBP Map. Right: Feature Histogram.

Note that in the Histogram of fig: 2, there are only 36 non-zero bins, that is, the feature. The y-axis of the Histogram in the fig 1 and fig 2 is the normalized count of the features in the given Original Image.

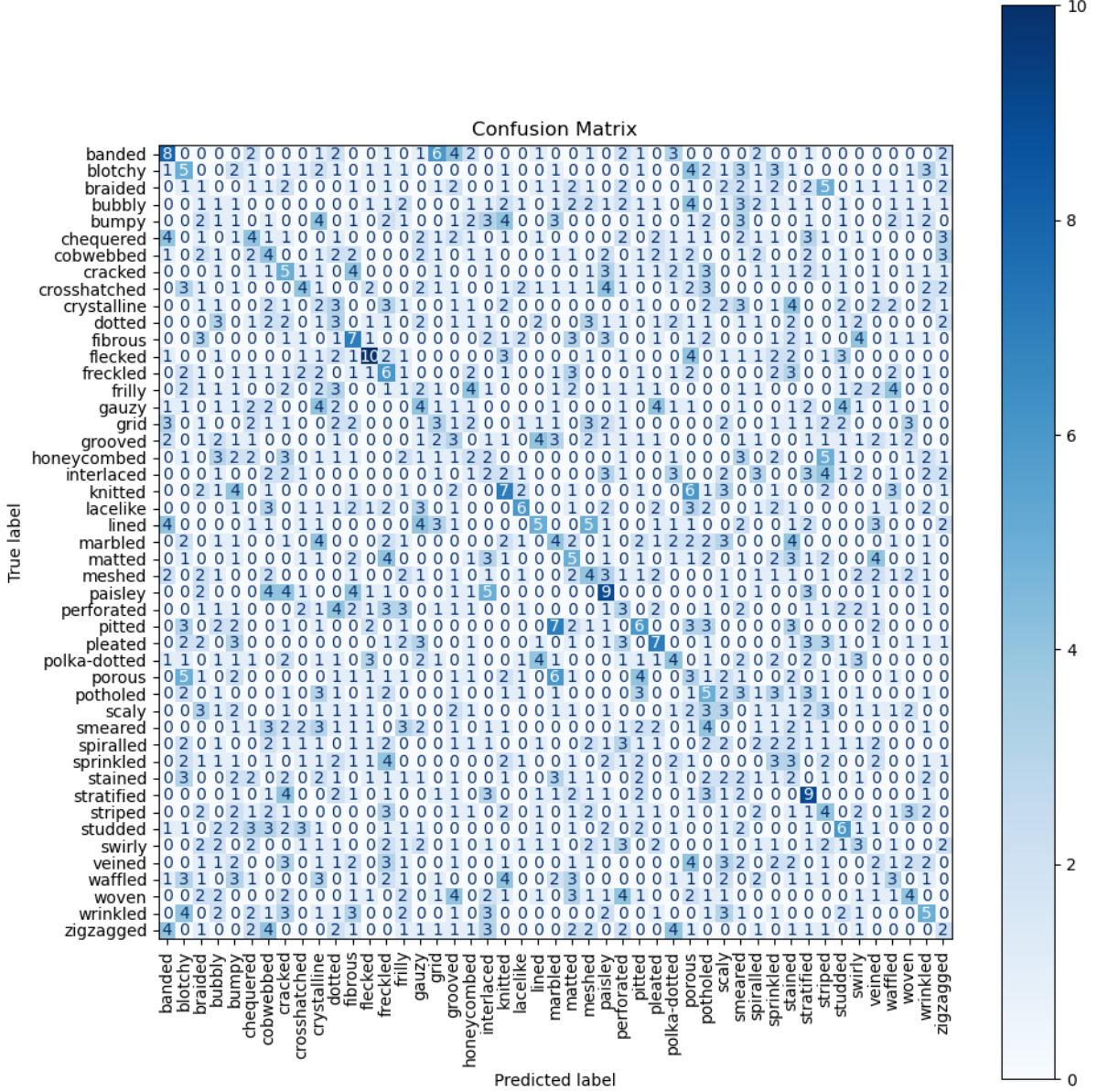


Figure 4: Confusion Matrix (Rotational Invariant LBP)

Why is accuracy low?

1. **High Cardinality:** Random guessing for 47 classes would yield $(1 \div 47) \sim 2.1\%$. Our model achieves $\sim 13.8\%$, which is better but still poor.
2. **Texture Similarity:** Some classes are visually similar. A simple histogram comparison (LBP) lacks the detail to distinguish them.

1.2.3 t-SNE Visualization (Class Separability)

To prove that the low accuracy is a data problem (class separability) rather than just a model failure, we visualized the feature space using t-SNE Plot as shown in Fig 5.

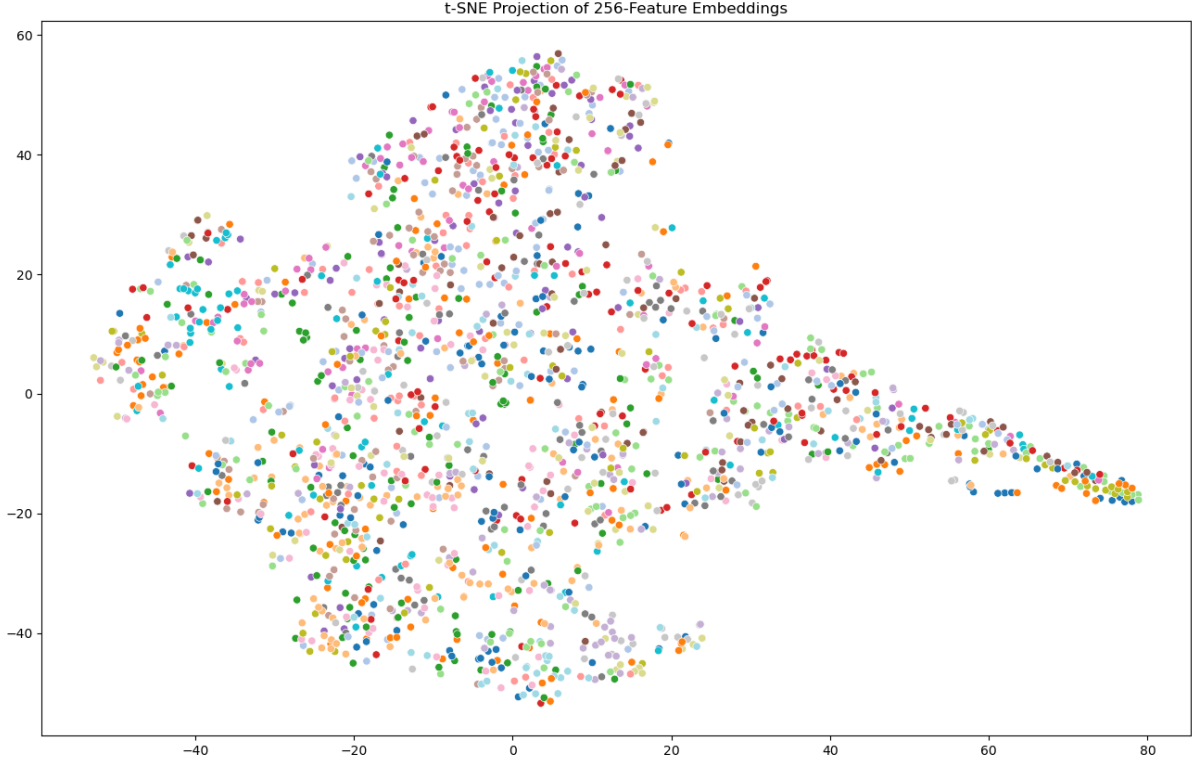


Figure 5: t-SNE Projection of LBP Features.

The plot shows a single dense blob rather than 47 distinct clusters. Each different colored small circle is a different class. This visually confirms that the classes are mathematically inseparable, explaining the low accuracy.

1.2.4 Performance on Rotated Images

We subjected the test set to rotations $\theta \in [0^\circ, 180^\circ]$. The accuracy for both models is recorded below in Table 1.

Table 1: **Accuracy (%) vs. Rotation Angle for All 47 Classes**

Angle ($^\circ$)	Standard LBP (SM)	Rotation Invariant (RI)
0	13.78	10.69
30	4.31	3.46
45	2.13	3.14
60	2.87	3.40
90	5.37	5.11
120	2.55	3.09
135	2.02	3.03
150	3.19	3.56
180	12.13	10.69

Key Observation from Table:

- **Standard LBP Failure:** Standard LBP drops from 13.78% to $\sim 2\%$ at intermediate angles ($45^\circ, 135^\circ$). The variance is too high.

- **Rotational Invariant Stability:** RI Model starts lower (10.69%) possibly due to the smaller number of features (36 in RI LBP vs 256 in Standard LBP), but it remains stable around 3.0 – 3.5% even at difficult angles. It does not suffer the same magnitude of variance as SM, but still the accuracy remains low.

1.2.5 Experiment on Randomized Subset

Since the entire dataset was difficult to separate, we selected a randomized subset of 5 classes to better observe rotation effects. The accuracy for both models is recorded below in Table 2.

Table 2: Accuracy (%) vs. Rotation Angle for a small subset of Classes

Angle (°)	Standard LBP (SM)	Rotation Invariant (RI)
0	41.50	41.50
30	19.00	20.50
45	21.50	23.50
60	22.50	17.50
90	27.50	34.50
120	27.50	26.50
135	21.50	27.50
150	27.50	26.00
180	55.50	58.00

1.2.6 Accuracy Comparison Plots

Visual Comparison of the accuracy trends of the Standard LBP Model and the Rotational Invariant LBP Model.

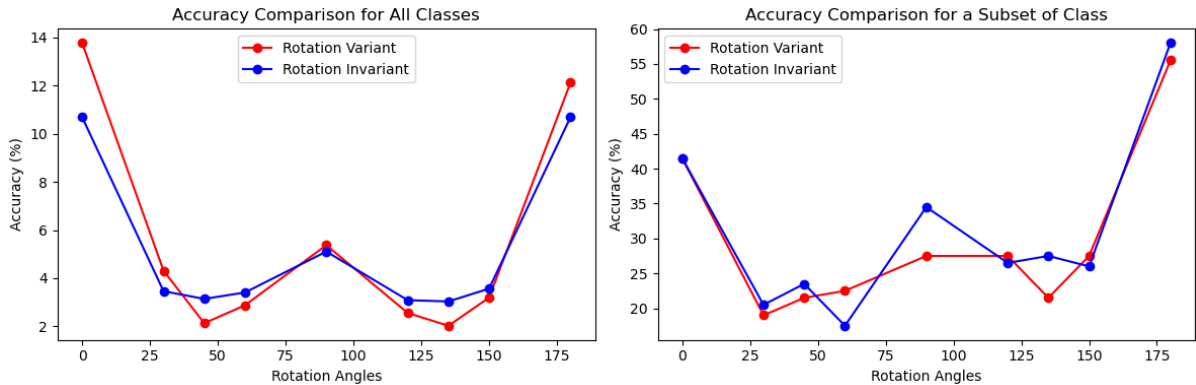


Figure 6: Accuracy Comparison.

In the Fig 6, On the left side shows the full dataset, Standard Model (Red) shows extreme volatility, dipping severely at oblique angles. The Rotational Invariant Model (Blue) is flatter. On the right is the subset of the 47 classes, we can see the trend is clearer: RI Model maintains a higher baseline at rotated angles (e.g., 30° – 150°) compared to Standard Model, validating its robustness.

2 Question 2: Adaptation to Sketch Domain

2.1 Methodology

2.1.1 Convolutional Neural Networks (CNNs)

For an input image I and a learnable kernel K , the feature map S is computed as:

$$S(i, j) = (I * K)(i, j) = \sum_m \sum_n I(m, n) K(i - m, j - n) \quad (3)$$

where (i, j) represents the spatial location.

2.1.2 Transfer Learning & Fine-Tuning

We use a pre-trained network, RESNET-18 (trained on ImageNet), as a feature extractor. We explore two primary models of Transfer Learning:

1. **Full Fine-Tuned Model:** All weights $\{W_{conv}, W_{fc}\}$ are updated based on the new Dataset.
2. **BatchNorm Fine-Tuned Model:** The weights W_{conv} of the convolutional base, except the BatchNorm layers are frozen (*requires_grad = False*), and the final fully connected layer weights W_{fc} are updated.

The optimization objective is to minimize the Cross-Entropy Loss $\mathcal{L}$ over the dataset $D = \{(x_i, y_i)\}_{i=1}^N$:

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log(\hat{y}_{i,c}) \quad (4)$$

where C is the number of classes, y is the ground truth, and $\hat{y}$ is the softmax probability.

2.2 Implementation Details

The implementation was conducted in Python using the PyTorch framework. The code structure is modularized as follows:

- **Data Loading:** Images were preprocessed using standard normalization $\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$ to match ImageNet statistics.
- **Model Architecture:** We employed the RESNET-18 model. The final fully connected layer was replaced to match the number of classes in our target dataset, ImageNet-Sketch.
- **Training Loop:** The optimization used Stochastic Gradient Descent (SGD). NUMBER_OF_EPOCHS used is 60, with BATCH_SIZE of 32 and LEARNING_RATE of 0.001

2.3 Analysis and Observations

2.3.1 Training Dynamics: Loss and Accuracy

The training process was monitored over 60 epochs. Figure 7 illustrates the convergence behavior of the Full Fine-tuned model.

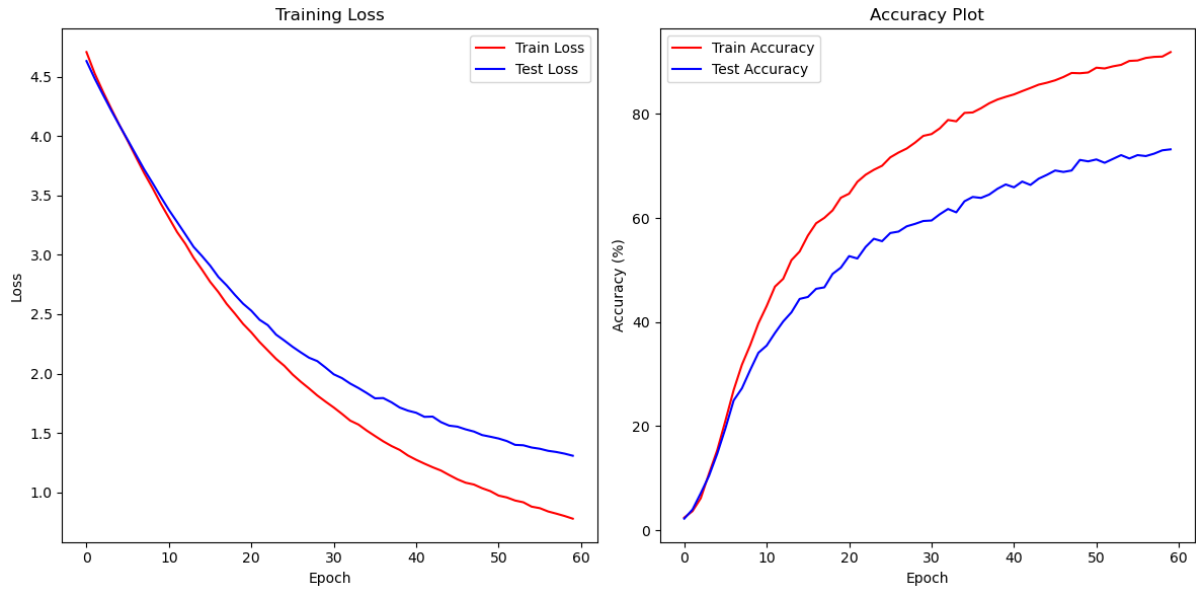


Figure 7: Left: Training vs. Testing Loss. Right: Training vs. Testing Accuracy.

Observation: We observe that the Training Loss (Red curve) decreases monotonically, indicating successful optimization. The Test Loss (Blue curve) stabilizes, suggesting the model is generalizing well without significant overfitting.

The Test Accuracy reaches **73.2%**, demonstrating that the ResNet Architecture, trained on the new dataset, successfully tunes its weights to correctly classify images in the ImageNet-Sketch dataset.

We typically expect a lower final loss because the deeper convolutional filters are allowed to adapt to the specific textures of the new dataset. The gap between Train and Test accuracy indicates the model's variance.

2.3.2 Early Layer Representations

Figure 8 compares the activations of the first convolution layer for both models. Remember that only 64 Feature Maps are shown for each layer for Convenience.

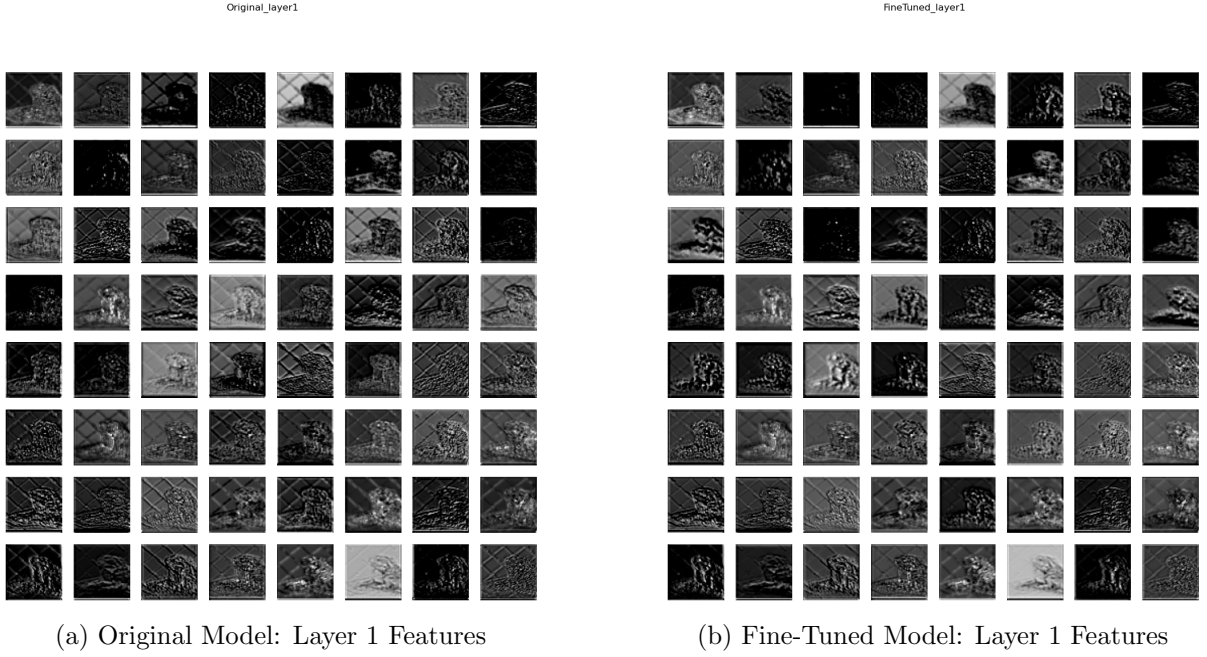


Figure 8: Early Layer Activations.

Analysis: The feature maps in the early layers primarily capture low-level features such as edges, gradients, and simple textures. Since these features are universal across image domains, both Model 1 and Model 2 show very similar activation patterns. This confirms the hypothesis that early layers in CNNs only detect blob-like features.

2.3.3 Middle Layer Representations

Figure 10 shows the activations from a deeper 2nd layer. Remember that only 64 Feature Maps are shown for each layer for Convenience.

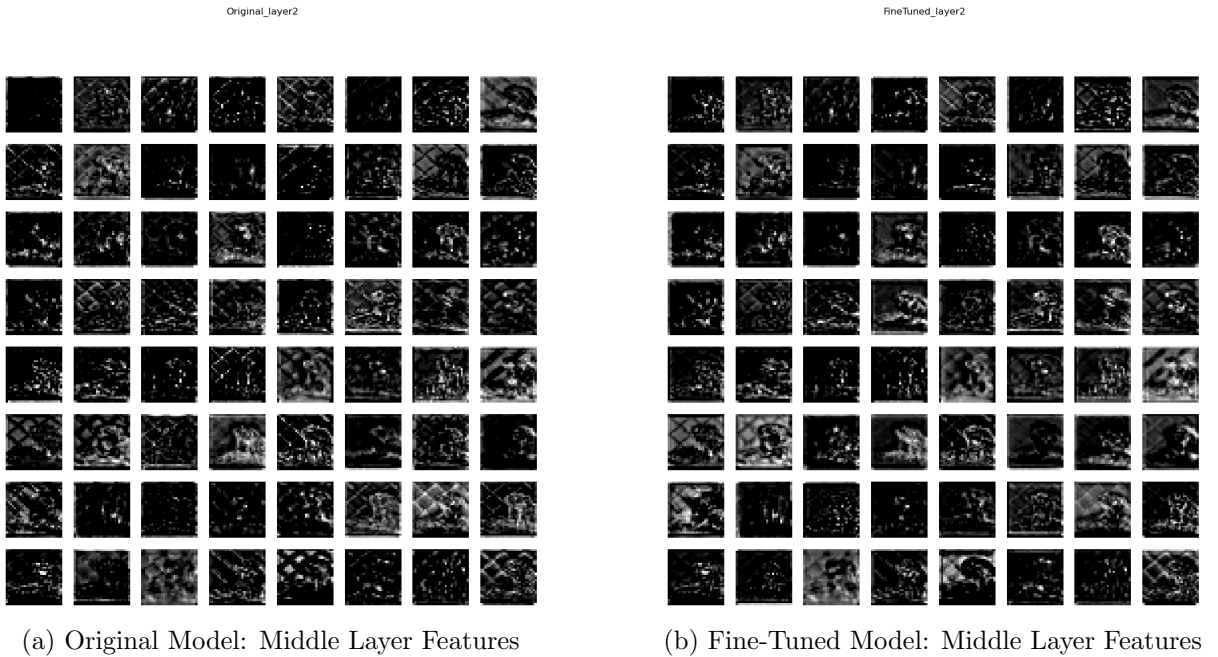


Figure 9: Mid Layer Activations

Analysis: In the middle layers, the network encodes mid-level semantic information (lines, textures, intensities, etc). The activations are slightly more detailed and reveal the texture of the objects of interest in the new dataset.

- **Original Model:** The activations look less specific to the target classes because the original ImageNet classes differ significantly from the target data.
- **Fine-Tuned Model:** The activations are slightly more detailed and reveal the texture of the objects of interest in the new dataset.

2.3.4 Deep Layer Representations

Figure 10 shows the activations from the last Convolution layer. Remember that only 64 Feature Maps are shown for each layer for Convenience.

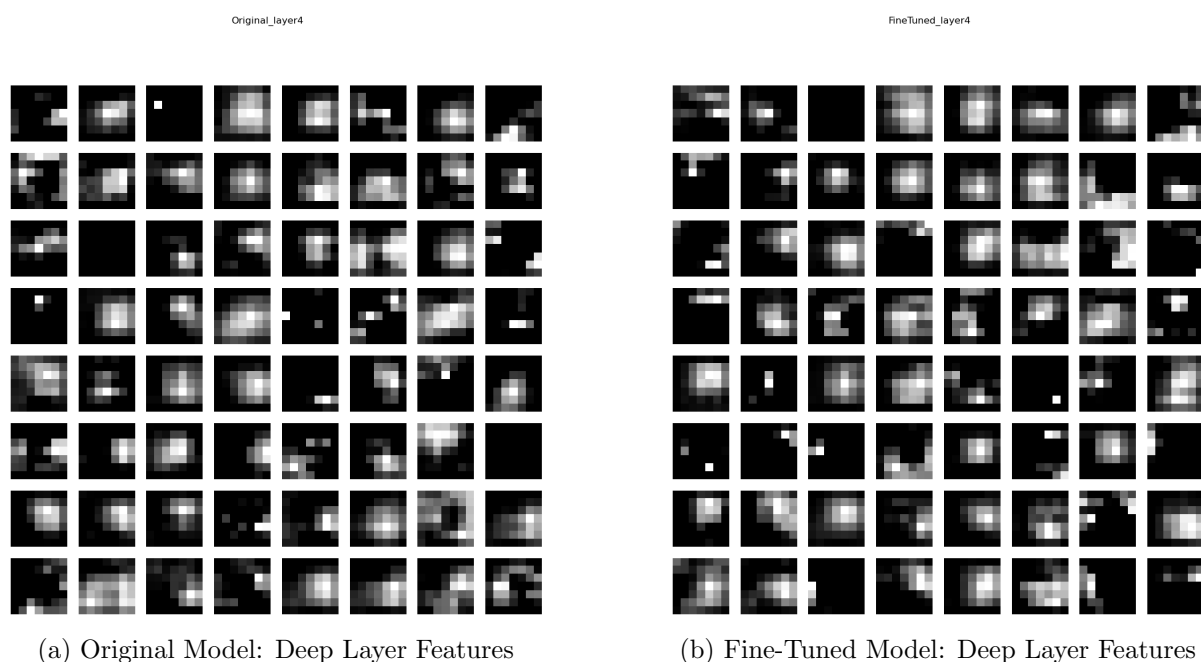


Figure 10: Deep Layer Activations

Analysis: In the deeper layers, the network encodes high-level semantic information (parts of objects, specific shapes). The activations are sharper and more localized around the objects of interest in the new dataset. There will be a significant difference between the Feature Maps of both Models.

2.3.5 Full Fine-Tuned vs BatchNorm Fine-Tuned: Loss and Accuracy

Our previous Fine-Tuned Model, in which all the weights were being updated, will be compared to the BatchNorm Fine-Tuned Model, in which, other than the final Fully-Connected Layer, all the BatchNorm2D layers are also updated. The training process for the BatchNorm Fine-Tuned Model was monitored over 60 epochs. Figure 11 illustrates the convergence behavior of this model.

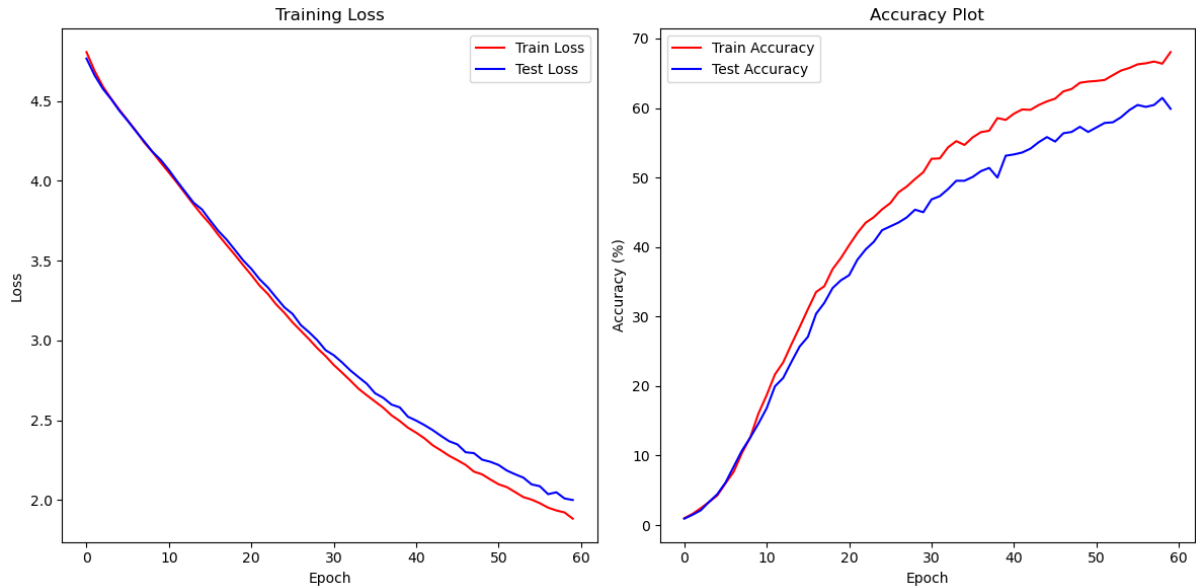


Figure 11: Left: Training vs. Testing Loss. Right: Training vs. Testing Accuracy.

Observation: We observe that the Training Loss (Red curve) decreases monotonically, indicating successful optimization. The Test Loss (Blue curve) stabilizes, suggesting the model is generalizing well without significant overfitting.

The Test Accuracy reaches **59.9%**, demonstrating that the ResNet Architecture, by updating only the BatchNorm layers and final Fully-Connected Layer, successfully leverages the feature maps generated by the Convolution Layers of the Original ResNet-18 Model to almost correctly classify images in the ImageNet-Sketch dataset.

2.4 Comparative Analysis of Fine-Tuning Strategies

Observation: The Full Fine-Tuning model (Model A) achieved a final test accuracy of **73.2%**, significantly outperforming the BatchNorm-only model (Model B), which saturated at **59.9%**. This result was expected because in the Full Fine-Tuned Model, the Convolution layers can update themselves to extract the meaning features based on the dataset, resulting in higher accuracy (by *approx.13%*) but also more training time for the same number of epochs (by *approx.10mins*), as compared to when we allow only the weights of BatchNorm layers to be updated.

The reason behind updating only the BatchNorm Layers is that the ImageNet-Sketch dataset involves Sketches of the objects. Sketches are grayscale with mostly white backgrounds, so updating the statistics of the data the model expects can also improve performance with less training time.

References

- [1] T. Ojala, M. Pietikainen, and T. Maenpaa, “Multiresolution gray-scale and rotation invariant texture classification with local binary patterns,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 24, no. 7, pp. 971–987, 2002.